

PZ-S1216F8-BD GPS 北斗定位模块开发手册

本手册我们将向大家介绍 PZ-S1216F8-BD GPS 北斗定位模块及其在普中 F407 最小系统&天马&麒麟开发板上的使用。本手册我们将使用 PZ-S1216F8-BD GPS 北斗定位模块实现 GPS/北斗定位。本章分为如下几部分内容：

- 1 PZ-S1216F8-BD GPS 北斗定位模块介绍
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 PZ-S1216F8-BD GPS 北斗定位模块介绍

1.1 特性参数

PZ-S1216F8-BD GPS/北斗模块是一款高性能 GPS/北斗双模定位模块。该模块主要特点包括：

- ①模块采用 S1216F8-BD 模组，体积小，性能优异。
- ②模块可通过串口进行各种参数设置，并可保存在内部 FLASH，使用方便。
- ③模块自带 IPX 接口，可以连接各种有源天线，建议连接 GPS/北斗双模有源天线。
- ④模块兼容 3.3V/5V 电平，方便连接各种单片机系统。
- ⑤模块自带可充电后备电池，可以掉电保持星历数据（在主电源断开后，后备电池可以维持半小时左右的 GPS/北斗星历数据的保存，以支持温启动或热启动，从而实现快速定位）。

模块通过串口与外部系统连接，串口波特率支持 4800、9600、19200、38400（默认）、57600、115200、230400 等不同速率，兼容 5V/3.3V 单片机系统，可以非常方便的与您的产品进行连接。该模块各参数如图所示：

项目	说明
接口特性	TTL，兼容 3.3V/5V 单片机系统
支持波特率	4800、9600、19200、38400（默认）、57600、115200、230400
接收特性	167 通道，支持 QZSS，WAAS，MSAS，EGNOS，GAGAN
定位精度	2.5 mCEP（SBAS：2.0mCEP）
更新速率	1/2/4/5/8/10/20 Hz
捕获时间	冷启动(1)：29S（最快） 温启动：27S 热启动：1S
冷启动灵敏度	-148dBm
捕获追踪灵敏度	-165dBm
通信协议	NMEA-0183 V3.01
工作温度	-40℃~85℃
模块尺寸	19.66mm*44.75mm

工作电压 (VCC)	DC3.3V~5.0V
工作电流	45mA
Voh	2.4V (Min)
Vol	0.4V (Max)
Vih	2V (Min)
Vil	0.8V (Max)
TXD/RXD 阻抗(2)	120欧

注(1)：冷启动是指模块所有保存的 GPS/北斗接收历史信息都丢失了（相当于主电源和后备电池都没电了），这种情况下重启，称之为冷启动。温启动是指模块保存了 GPS/北斗接收历史信息，但是当前可视卫星的信息和保存的信息不一致了，这样的条件下重启，称之为温启动。热启动则是指在模块保存了 GPS/北斗接收历史信息且与当前可视卫星信息一致，这样的条件下重启，称之为热启动。

注(2)：模块的 TXD 和 RXD 脚内部接了 120 欧电阻，做输出电平兼容处理，所以在使用的时候要注意，导线电阻不可过大(尤其是接 USB 转 TTL 串口模块的时候,如果模块的 TXD、RXD 上带了 LED，那就会有问题)，否则可能导致通信不正常。

PZ-S1216F8-BD GPS/北斗模块支持多种通信波特率，通过串口进行设置，并可以保存在模块内部 FLASH，模块默认波特率为：38400（8 位数据位，1 位停止位，无奇偶校验），详细的设置方法，我们会在后面介绍。

1.2 模块说明

1.2.1 模块引脚说明

PZ-S1216F8-BD GPS/北斗模块非常小巧（19.66mm*44.75mm），模块通过 1 个 2*4 的间距为 2.54mm 的排针与外部连接，模块上有 4 个安装孔，方便大家安装到自己的设备里面，模块外观如图所示：



从图中可以看到，从右到左依次为模块引出的 8 个脚，其中 NC 代表未与模块相连即待扩展脚，可用的管脚只有 5 个，各引脚的详细描述如图所示：

管脚名称	功能说明
VCC	电源（3.3V~5.0V）
GND	地
TXD	模块串口发送脚（TTL 电平，不能直接接 RS232 电平！）， 可接单片机的 RXD
RXD	模块串口接收脚（TTL 电平，不能直接接 RS232 电平！）， 可接单片机的 TXD
PPS	时钟脉冲输出脚

其中 PPS 引脚同时连接到了模块自带的状态指示灯：PPS1，该引脚连接在 S1216F8-BD 模组的 1PPS 端口，该端口的输出特性可以通过程序设置。PPS1 指示灯（即 PPS 引脚），在默认条件下（没经过程序设置），有 2 个状态：

- 1，常亮，表示模块已开始工作，但还未实现定位。
- 2，闪烁（100ms 灭，900ms 亮），表示模块已经定位成功。

这样，通过 PPS1 指示灯，我们就可以很方便的判断模块的当前状态，方便大家使用。

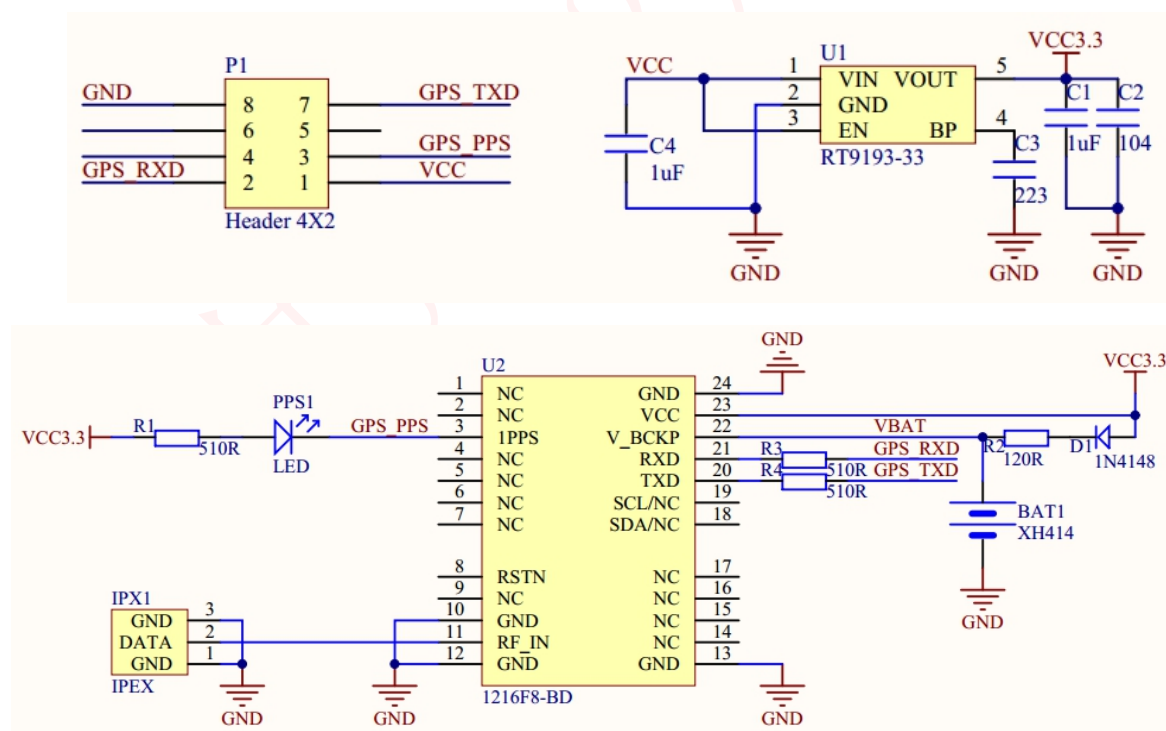
另外，模块的右下角的 IPX1 接口，用来外接一个有源天线，通过外接有源天线，我们就可以把模块放到室内，天线放到室外，实现室内定位。

一般 GPS 有源天线都是采用 SMA 接口，我们需要准备一根 IPX（IPEX）转 SMA 的连接线，从而连接 PZ-S1216F8-BD GPS/北斗模块与有源天线，如图所示：



PZ-S1216F8-BD GPS/北斗模块默认采用 NMEA-0183 协议输出 GPS/北斗定位数据，并可以通过 SkyTraQ 协议对模块进行配置，NMEA-0183 协议将在后面会详细介绍，SkyTraQ 配置协议，请参考《Binary Messages of SkyTraQ Venus 8 GNSSReceiver.pdf》。

使用 PZ-S1216F8-BD GPS/北斗模块，任何单片机（3.3V/5V 电源）都可以很方便的实现 GPS/北斗定位。PZ-S1216F8-BD GPS/北斗模块的原理图如图所示：



1.2.2 模块使用说明

PZ-S1216F8-BD GPS/北斗模块同外部设备的通信接口采用 UART（串口）方式，输出的 GPS/北斗定位数据采用 NMEA-0183 协议（默认），控制协议为

SkyTraq 协议（该协议的详细介绍请看 Binary Messages of SkyTraq Venus 8 GNSS Receiver.pdf 这个文档）。这里我们主要向大家介绍 NMEA-0183 协议。

（1）NMEA-0183 协议简介

NMEA 0183 是美国国家海洋电子协会（National Marine Electronics Association）为海用电子设备制定的标准格式。目前业已成了 GPS/北斗导航设备统一的 RTCM（Radio Technical Commission for Maritime services）标准协议。

NMEA-0183 协议采用 ASCII 码来传递 GPS 定位信息，我们称之为帧。

帧格式形如：\$acccc,ddd,ddd,...,ddd*hh(CR)(LF)

- 1、“\$”：帧命令起始位
- 2、aaaa：地址域，前两位为识别符（aa），后三位为语句名（ccc）
- 3、ddd...ddd：数据
- 4、“*”：校验和前缀（也可以作为语句数据结束的标志）
- 5、hh：校验和（check sum），\$与*之间所有字符 ASCII 码的校验和（各字节做异或运算，得到校验和后，再转换 16 进制格式的 ASCII 字符）
- 6、(CR)(LF)：帧结束，回车和换行符

NMEA-0183 常用命令如图所示：

序号	命令	说明	最大帧长
1	\$GNGGA	GPS/北斗定位信息	72
2	\$GNGSA	当前卫星信息	65
3	\$GPGSV	可见 GPS 卫星信息	210
4	\$BDGSV	可见北斗卫星信息	210
5	\$GNRMC	推荐定位信息	70
6	\$GNVTG	地面速度信息	34
7	\$GNGLL	大地坐标信息	--
8	\$GNZDA	当前时间(UTC ¹)信息	--

注 1：即协调世界时，相当于本初子午线（0 度经线）上的时间，北京时间比 UTC 早 8 个小时。

接下来我们分别介绍这些命令。

1. \$GNGGA（GPS 定位信息，Global Positioning System Fix Data）

\$GNGGA 语句的基本格式如下（其中 M 指单位 M，hh 指校验和，CR 和 LF 代表回车换行，下同）：

`$GNGGA, (1), (2), (3), (4), (5), (6), (7), (8), (9), M, (10), M, (11), (12)*hh(CR) (LF)`

- (1) UTC 时间，格式为 hhmmss.ss；
- (2) 纬度，格式为 ddmm.mmmmm（度分格式）；
- (3) 纬度半球，N 或 S（北纬或南纬）；
- (4) 经度，格式为 dddmm.mmmmm（度分格式）；
- (5) 经度半球，E 或 W（东经或西经）；
- (6) GPS 状态，0=未定位，1=非差分定位，2=差分定位；
- (7) 正在使用的用于定位的卫星数量（00~12）
- (8) HDOP 水平精确度因子（0.5~99.9）
- (9) 海拔高度（-9999.9 到 9999.9 米）
- (10) 大地水准面高度（-9999.9 到 9999.9 米）
- (11) 差分时间（从最近一次接收到差分信号开始的秒数，非差分定位，此项为空）
- (12) 差分参考基站标号（0000 到 1023，首位 0 也将传送，非差分定位，此项为空）

例如：

`$GNGGA, 095528.000, 2318.1133, N, 11319.7210, E, 1, 06, 3.7, 55.1, M, -5.4, M, , 0000*69`

2. \$GNGSA（当前卫星信息）

\$GNGSA 语句的基本格式如下：

`$GNGSA, (1), (2), (3), (3), (3), (3), (3), (3), (3), (3), (3), (3), (3), (3), (4), (5), (6)*
hh(CR) (LF)`

- (1) 模式，M = 手动，A = 自动。
- (2) 定位类型，1=未定位，2=2D 定位，3=3D 定位。
- (3) 正在用于定位的卫星号（01~32）
- (4) PDOP 综合位置精度因子（0.5~99.9）
- (5) HDOP 水平精度因子①（0.5~99.9）
- (6) VDOP 垂直精度因子（0.5~99.9）

例如：

```
$GNGSA, A, 3, 14, 22, 24, 12, , , , , , , , 4. 2, 3. 7, 2. 1*2D
```

```
$GNGSA, A, 3, 209, 214, , , , , , , , 4. 2, 3. 7, 2. 1*21
```

注①： 精度因子值越小，则准确度越高。

3. \$GPGSV (可见卫星数, GPS Satellites in View)

\$GPGSV 语句的基本格式如下：

```
$GPGSV, (1), (2), (3), (4), (5), (6), (7), ..., (4), (5), (6), (7)*hh(CR) (LF)
```

- (1) GSV 语句总数。
- (2) 本句 GSV 的编号。
- (3) 可见卫星的总数 (00~12, 前面的 0 也将被传输)。
- (4) 卫星编号 (01~32, 前面的 0 也将被传输)。
- (5) 卫星仰角 (00~90 度, 前面的 0 也将被传输)。
- (6) 卫星方位角 (000~359 度, 前面的 0 也将被传输)。
- (7) 信噪比 (00~99dB, 没有跟踪到卫星时为空)。

注： 每条 GSV 语句最多包括四颗卫星的信息，其他卫星的信息将在下一条 \$GPGSV 语句中输出。

例如：

```
$GPGSV, 3, 1, 11, 18, 73, 129, 19, 10, 71, 335, 40, 22, 63, 323, 41, 25, 49, 127, 06*78
```

```
$GPGSV, 3, 2, 11, 14, 41, 325, 46, 12, 36, 072, 34, 31, 32, 238, 22, 21, 23, 194, 08*76
```

```
$GPGSV, 3, 3, 11, 24, 21, 039, 40, 20, 08, 139, 07, 15, 08, 086, 03*45
```

4. \$BDGSV (可见卫星数, GPS Satellites in View)

\$BDGSV 语句的基本格式如下：

```
$BDGSV, (1), (2), (3), (4), (5), (6), (7), ..., (4), (5), (6), (7)*hh(CR) (LF)
```

- (1) GSV 语句总数。
- (2) 本句 GSV 的编号。
- (3) 可见卫星的总数 (00~12, 前面的 0 也将被传输)。
- (4) 卫星编号 (01~32, 前面的 0 也将被传输)。
- (5) 卫星仰角 (00~90 度, 前面的 0 也将被传输)。
- (6) 卫星方位角 (000~359 度, 前面的 0 也将被传输)。

(7) 信噪比 (00~99dB, 没有跟踪到卫星时空) 。

注: 每条 GSV 语句最多包括四颗卫星的信息, 其他卫星的信息将在下一条 \$BDGSV 语句中输出。

例如:

\$BDGSV, 1, 1, 02, 209, 64, 354, 40, 214, 05, 318, 40*69

5. \$GNRMC (推荐定位信息, Recommended Minimum Specific GPS/Transit Data)

\$GNRMC 语句的基本格式如下:

\$GNRMC, (1), (2), (3), (4), (5), (6), (7), (8), (9), (10), (11), (12)*hh(CR) (LF)

- (1) UTC 时间, hhmmss (时分秒)
- (2) 定位状态, A=有效定位, V=无效定位
- (3) 纬度 ddmm.mmmmm (度分)
- (4) 纬度半球 N (北半球) 或 S (南半球)
- (5) 经度 dddmm.mmmmm (度分)
- (6) 经度半球 E (东经) 或 W (西经)
- (7) 地面速率 (000.0~999.9 节)
- (8) 地面航向 (000.0~359.9 度, 以真北方为参考基准)
- (9) UTC 日期, ddmmyy (日月年)
- (10) 磁偏角 (000.0~180.0 度, 前导位数不足则补 0)
- (11) 磁偏角方向, E (东) 或 W (西)
- (12) 模式指示 (A=自主定位, D=差分, E=估算, N=数据无效)

例如:

\$GNRMC, 095554.000, A, 2318.1327, N, 11319.7252, E, 000.0, 005.7, 081215,,, A*73

6. \$GNVTG (地面速度信息, Track Made Good and Ground Speed)

\$GNVTG 语句的基本格式如下:

\$GNVTG, (1), T, (2), M, (3), N, (4), K, (5)*hh(CR) (LF)

- (1) 以真北为参考基准的地面航向 (000~359 度, 前面的 0 也将被传输)
- (2) 以磁北为参考基准的地面航向(000~359 度, 前面的 0 也将被传输)
- (3) 地面速率(000.0~999.9 节, 前面的 0 也将被传输)

(4) 地面速率 (0000.0~1851.8 公里/小时, 前面的 0 也将被传输)

(5) 模式指示 (A=自主定位, D=差分, E=估算, N=数据无效)

例如:

`$GNVTG,005.7,T,,M,000.0,N,000.0,K,A*11`

7. \$GNGLL (定位地理信息, Geographic Position)

\$GNGLL 语句的基本格式如下:

`$GNGLL, (1), (2), (3), (4), (5), (6), (7)*hh (CR) (LF)`

(1) 纬度 ddmm.mmmmm (度分)

(2) 纬度半球 N (北半球) 或 S (南半球)

(3) 经度 dddmm.mmmmm (度分)

(4) 经度半球 E (东经) 或 W (西经)

(5) UTC 时间: hhmmss (时分秒)

(6) 定位状态, A=有效定位, V=无效定位

(7) 模式指示 (A=自主定位, D=差分, E=估算, N=数据无效)

例如:

`$GNGLL,2318.1330,N,11319.7250,E,095556.000,A,A*4F`

8. \$GNZDA (当前时间信息)

\$GNZDA 语句的基本格式如下:

`$GNZDA, (1), (2), (3), (4), (5), (6)*hh (CR) (LF)`

(1) UTC 时间: hhmmss (时分秒)

(2) 日

(3) 月

(4) 年

(5) 本地区域小时 (NEO-6M 未用到, 为 00)

(6) 本地区域分钟 (NEO-6M 未用到, 为 00)

例如:

`$GNZDA,095555.000,08,12,2015,00,00*4C`

NMEA-0183 协议命令帧部分就介绍到这里, 接下来我们看看 NMEA-0183 协议的校验, 通过前面的介绍, 我们知道每一帧最后都有一个 hh 的校验和, 该校

验和是通过计算\$与*之间所有字符 ASCII 码的异或运算得到，将得到的结果以 ASCII 字符表示就是该校验（hh）。

例如语句：`$GNZDA,095555.000,08,12,2015,00,00*4C`，校验和（红色部分参与计算）计算方法为：

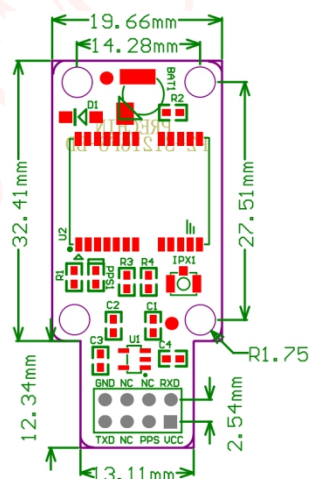
```
0X47 xor 0X4E xor 0X5A xor 0X44 xor 0X41 xor 0X2C xor 0X30 xor 0X39
xor 0X35 xor 0X35 xor 0X35 xor 0X35 xor 0X2E xor 0X30 xor 0X30 xor 0X30
xor 0X2C xor 0X30 xor 0X38 xor 0X2C xor 0X31 xor 0X32 xor 0X2C xor 0X32
xor 0X30 xor 0X31 xor 0X35 xor 0X2C xor 0X30 xor 0X30 xor 0X2C xor 0X30
xor 0X30
```

得到的结果就是 0X4C，用 ASCII 表示就是 4C。

NMEA-0183 协议我们就介绍到这里，了解了该协议，我们就可以编写单片机代码，解析 NMEA-0183 数据，从而得到 GPS/北斗定位的各种信息了。

1.3 结构尺寸

PZ-S1216F8-BD GPS 模块尺寸结构如下图所示：



2 硬件设计

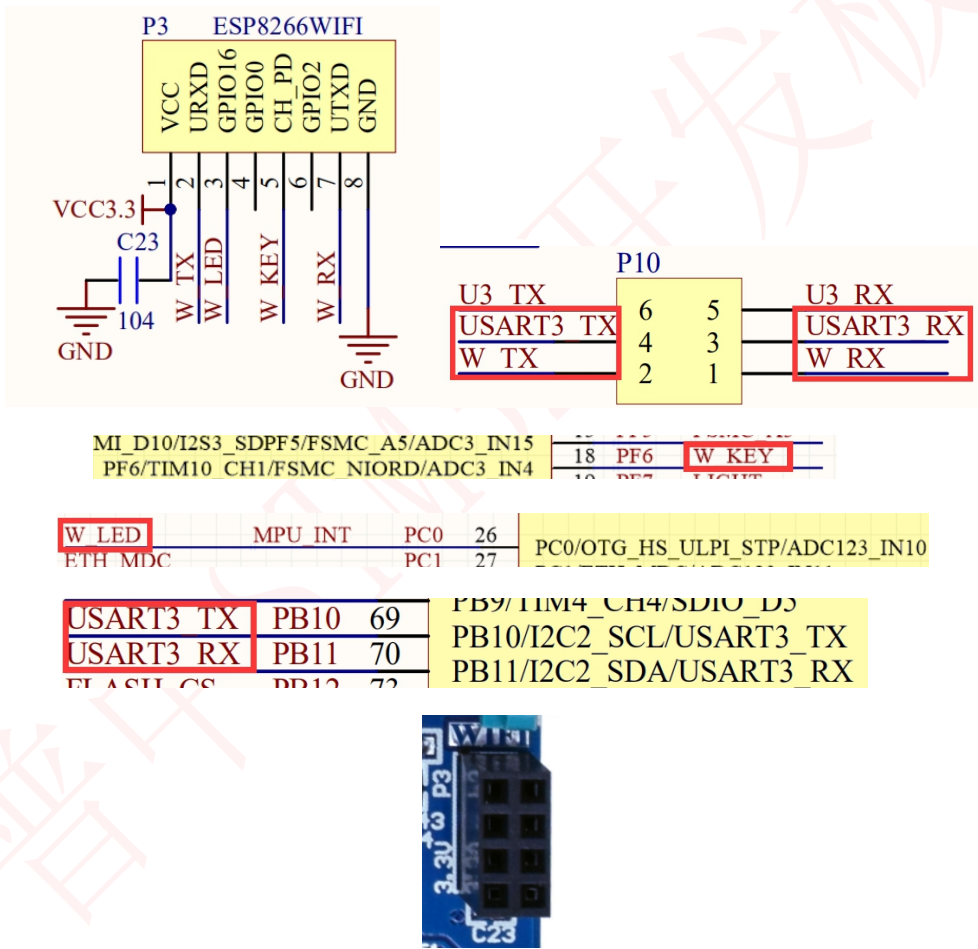
2.1 硬件准备

本实验所需要的硬件资源如下：

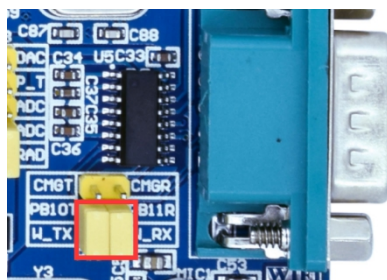
- ①F407 最小系统&天马&麒麟开发板 1 个
- ②TFTLCD 模块（不用彩屏也可用串口输出）
- ③PZ-S1216F8-BD GPS 模块 1 个
- ④USB 线一条（用于供电和模块与电脑串口调试助手通信）

2.2 模块与开发板连接

PZ-S1216F8-BD GPS 模块可直接与天马&麒麟开发板板载的 WIFI&蓝牙模块接口（P3）进行连接，该接口与 MCU 连接原理图（以麒麟为例）如下图所示：



从上图看出,WIFI 接口与 STM32 使用时,必须将 P10 的 USART3_TX(PB10)和 W_TX 以及 USART3_RX (PB11)和 W_RX 连接,才能完成和 STM32 的连接。默认天马&麒麟的 P10 端子上有黄色跳线帽,直接短接到对应端即可。如下图所示:



对于 F407 最小系统，是没有 WIFI 接口的，所以需要通过杜邦线将模块管脚与上述 STM32 的 IO 口连接即可。

PZ-S1216F8-BD GPS 模块与 STM32 的 IO 口连接关系如下：(PZ-S1216F8-BD GPS→STM32)

```
VCC→3.3V
GND→GND
RXD→PB10
TXD→PB11
PPS→PC0
```

注意：使用天马&麒麟开发板连接模块时，模块插入 WIFI 接口处请注意方向，可将模块电源与 WIFI 接口丝印电源对应即可。

注意：我们虽然将 PPS 连接到了开发板的 PC0 脚，但是本例程并没有用到 PPS 信号，所以没做处理。

3 软件设计

本实验所实现的功能为：通过串口 3 连接 PZ-S1216F8-BD GPS/北斗模块，然后通过液晶显示 GPS/北斗信息，包括精度、纬度、高度、速度、用于定位的卫星数、可见卫星数、UTC 时间等信息。另外通过 KEY_UP 按键，可以开启或关闭 NMEA 数据的上传（即输出到串口 1，方便开发调试）。

我们打开本实验工程，可以看到我们的工程 APP 列表中多了 usart3.c 和 gps.c 源文件，以及头文件 usart3.h、gps.h。

3.1 usart3.c

首先，我们来看 usart3.c 里面代码，如下：

```
1. #include "usart3.h"
```

```

2.  #include "stdarg.h"
3.  #include "stdio.h"
4.  #include "string.h"
5.  #include "time.h"
6.
7.
8.  //串口接收缓存区
9.  u8 USART3_RX_BUF[USART3_MAX_RECV_LEN]; //接收缓冲,最大 USART3_MAX_RECV_LEN 个字节.
10. u8  USART3_TX_BUF[USART3_MAX_SEND_LEN]; //发送缓冲,最大 USART3_MAX_SEND_LEN 字节
11.
12.
13. //通过判断接收连续 2 个字符之间的时间差不大于 10ms 来决定是不是一次连续的数据.
14. //如果 2 个字符接收间隔超过 10ms,则认为不是 1 次连续数据.也就是超过 10ms 没有接收到
15. //任何数据,则表示此次接收完毕.
16. //接收到的数据状态
17. //[15]:0,没有接收到数据;1,接收到了一批数据.
18. //[14:0]:接收到的数据长度
19. u16 USART3_RX_STA=0;
20.
21. void USART3_Init(u32 bound)
22. {
23.
24.     GPIO_InitTypeDef GPIO_InitStructure;
25.     USART_InitTypeDef USART_InitStructure;
26.     NVIC_InitTypeDef NVIC_InitStructure;
27.
28.     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB,ENABLE); //使能 GPIOA 时钟
29.     RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART3,ENABLE); //使能 USART3 时钟
30.
31.     //串口 3 对应引脚复用映射
32.     GPIO_PinAFConfig(GPIOB,GPIO_PinSource10,GPIO_AF_USART3);
33.     GPIO_PinAFConfig(GPIOB,GPIO_PinSource11,GPIO_AF_USART3);
34.
35.     //USART3 端口配置
36.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10|GPIO_Pin_11 ;
37.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF; //复用功能
38.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; //速度 50MHz
39.     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; //推挽复用输出
40.     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP; //上拉
41.     GPIO_Init(GPIOB,&GPIO_InitStructure);
42.
43.     //Usart3 NVIC 配置
44.     NVIC_InitStructure.NVIC_IRQChannel = USART3_IRQn;
45.     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=2 ; //抢占优先级 0

```

```

46.     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3;           //子优先级 0
47.     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;           //IRQ 通道使能
48.     NVIC_Init(&NVIC_InitStructure); //根据指定的参数初始化 VIC 寄存器
49.
50.     //USART3 初始化设置
51.     USART_InitStructure.USART_BaudRate = bound; //串口波特率
52.     USART_InitStructure.USART_WordLength = USART_WordLength_8b; //字长为 8 位数据格式
53.     USART_InitStructure.USART_StopBits = USART_StopBits_1; //一个停止位
54.     USART_InitStructure.USART_Parity = USART_Parity_No; //无奇偶校验位
55.     USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //无硬件
    数据流控制
56.     USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模式
57.     USART_Init(USART3, &USART_InitStructure); //初始化串口 3
58.
59.     USART_ITConfig(USART3, USART_IT_RXNE, ENABLE); //开启串口接收中断
60.
61.     USART_Cmd(USART3, ENABLE); //使能串口 3
62.
63.     TIM7_Int_Init(100-1, 8400-1); //10ms 中断一次
64.     USART3_RX_STA=0; //清零
65.     TIM_Cmd(TIM7, DISABLE); //关闭定时器 7
66. }
67.
68. void USART3_IRQHandler( void )
69. {
70.     u8 res;
71.     if(USART_GetITStatus(USART3, USART_IT_RXNE) != RESET) //接收到数据
72.     {
73.         res =USART_ReceiveData(USART3);
74.         if((USART3_RX_STA&(1<<15))==0) //接收完的一批数据,还没有被处理,则不再接收其他数据
75.         {
76.             if(USART3_RX_STA<USART3_MAX_RECV_LEN) //还可以接收数据
77.             {
78.                 TIM_SetCounter(TIM7,0); //计数器清空 //计数器清空
79.                 if(USART3_RX_STA==0) //使能定时器 7 的中断
80.                 {
81.                     TIM_Cmd(TIM7, ENABLE); //使能定时器 7
82.                 }
83.                 USART3_RX_BUF[USART3_RX_STA++]=res; //记录接收到的值
84.             }
85.             else
86.             {
87.                 USART3_RX_STA|=1<<15; //强制标记接收完成
88.             }

```



```

89.     }
90. }
91. }
92.
93.
94. //串口 3,printf 函数
95. //确保一次发送数据不超过 USART3_MAX_SEND_LEN 字节
96. void usart3_printf(char* fmt,...)
97. {
98.     u16 i,j;
99.     va_list ap;
100.    va_start(ap,fmt);
101.    vsprintf((char*)USART3_TX_BUF,fmt,ap);
102.    va_end(ap);
103.    i=strlen((const char*)USART3_TX_BUF); //此次发送数据的长度
104.    for(j=0;j<i;j++) //循环发送数据
105.    {
106.        while(USART_GetFlagStatus(USART3,USART_FLAG_TC)==RESET); //循环发送,直到发送完毕
107.        USART_SendData(USART3,USART3_TX_BUF[j]);
108.    }
109. }

```

这部分代码主要实现了串口 3 的初始化,以及实现了串口 3 的 printf 函数: u3_printf, 和串口 3 的接收处理。串口 3 的数据接收,采用了定时判断的方法,对于一次连续接收的数据,如果出现连续 10ms 没有接收到任何数据,则表示这次连续接收数据已经结束。

usart3.h 里面的代码我们就不在这里列出了,请大家参考对应源码。

3.2 gps.c

下面我们来看下 gps.c 里面的代码,如下:

```

1.  #include "gps.h"
2.  #include "led.h"
3.  #include "SysTick.h"
4.  #include "usart3.h"
5.  #include "stdio.h"
6.  #include "stdarg.h"
7.  #include "string.h"
8.  #include "math.h"
9.
10.
11.

```

```

12.
13.     const u32 BAUD_id[9]={4800,9600,19200,38400,57600,115200,230400,460800,921600}; //模块支持波特
    率数组
14.
15.     //从 buf 里面得到第 cx 个逗号所在的位置
16.     //返回值:0~0XFE,代表逗号所在位置的偏移.
17.     //         0XFF,代表不存在第 cx 个逗号
18.     u8 NMEA_Comma_Pos(u8 *buf,u8 cx)
19.     {
20.         u8 *p=buf;
21.         while(cx)
22.         {
23.             if(*buf=='*' || *buf<' ' || *buf>'z')return 0XFF; //遇到 '*' 或者非法字符,则不存在第 cx 个逗
    号
24.             if(*buf==',')cx--;
25.             buf++;
26.         }
27.         return buf-p;
28.     }
29.
30.     //m^n 函数
31.     //返回值:m^n 次方.
32.     u32 NMEA_Pow(u8 m,u8 n)
33.     {
34.         u32 result=1;
35.         while(n-->0)result*=m;
36.         return result;
37.     }
38.
39.     //str 转换为数字,以','或者'*'结束
40.     //buf:数字存储区
41.     //dx:小数点位数,返回给调用函数
42.     //返回值:转换后的数值
43.     int NMEA_Str2num(u8 *buf,u8*dx)
44.     {
45.         u8 *p=buf;
46.         u32 ires=0,fres=0;
47.         u8 ilen=0,flen=0,i;
48.         u8 mask=0;
49.         int res;
50.         while(1) //得到整数和小数的长度
51.         {
52.             if(*p=='-'){mask|=0X02;p++;} //是负数
53.             if(*p==',' || (*p=='*'))break; //遇到结束了

```

```

54.         if(*p=='.'){mask|=0X01;p++;} //遇到小数点了
55.         else if(*p>'9' || (*p<'0')) //有非法字符
56.         {
57.             ilen=0;
58.             flen=0;
59.             break;
60.         }
61.         if(mask&0X01)flen++;
62.         else ilen++;
63.         p++;
64.     }
65.     if(mask&0X02)buf++; //去掉负号
66.     for(i=0;i<ilen;i++) //得到整数部分数据
67.     {
68.         ires+=NMEA_Pow(10,ilen-1-i)*(buf[i]-'0');
69.     }
70.     if(flen>5)flen=5; //最多取 5 位小数
71.     *dx=flen; //小数点位数
72.     for(i=0;i<flen;i++) //得到小数部分数据
73.     {
74.         fres+=NMEA_Pow(10,flen-1-i)*(buf[ilen+1+i]-'0');
75.     }
76.     res=ires*NMEA_Pow(10,flen)+fres;
77.     if(mask&0X02)res=-res;
78.     return res;
79. }
80.
81. //分析 GPGSV 信息
82. //gpsx: nmea 信息结构体
83. //buf: 接收到的 GPS 数据缓冲区首地址
84. void NMEA_GPGSV_Analysis(nmea_msg *gpsx, u8 *buf)
85. {
86.     u8 *p, *p1, dx;
87.     u8 len, i, j, slx=0;
88.     u8 posx;
89.     p=buf;
90.     p1=(u8*)strstr((const char *)p, "$GPGSV");
91.     len=p1[7]-'0'; //得到 GPGSV 的条数
92.     posx=NMEA_Comma_Pos(p1, 3); //得到可见卫星总数
93.     if(posx!=0XFF)gpsx->svnum=NMEA_Str2num(p1+posx, &dx);
94.     for(i=0;i<len;i++)
95.     {
96.         p1=(u8*)strstr((const char *)p, "$GPGSV");
97.         for(j=0;j<4;j++)

```

```

98.         {
99.             posx=NMEA_Comma_Pos(p1,4+j*4);
100.            if(posx!=0XFF)gpsx->s1msg[slx].num=NMEA_Str2num(p1+posx,&dx);    //得到卫星编号
101.            else break;
102.            posx=NMEA_Comma_Pos(p1,5+j*4);
103.            if(posx!=0XFF)gpsx->s1msg[slx].eledeg=NMEA_Str2num(p1+posx,&dx);//得到卫星仰角
104.            else break;
105.            posx=NMEA_Comma_Pos(p1,6+j*4);
106.            if(posx!=0XFF)gpsx->s1msg[slx].azideg=NMEA_Str2num(p1+posx,&dx);//得到卫星方位
            角
107.            else break;
108.            posx=NMEA_Comma_Pos(p1,7+j*4);
109.            if(posx!=0XFF)gpsx->s1msg[slx].sn=NMEA_Str2num(p1+posx,&dx); //得到卫星信噪比
110.            else break;
111.            slx++;
112.        }
113.        p=p1+1;//切换到下一个 GPGSV 信息
114.    }
115. }
116.
117. //分析 BDGSV 信息
118. //gpsx:nmea 信息结构体
119. //buf:接收到的 GPS 数据缓冲区首地址
120. void NMEA_BDGSV_Analysis(nmea_msg *gpsx,u8 *buf)
121. {
122.     u8 *p,*p1,dx;
123.     u8 len,i,j,slx=0;
124.     u8 posx;
125.     p=buf;
126.     p1=(u8*)strstr((const char *)p,"$BDGSV");
127.     len=p1[7]-'0';    //得到 BDGSV 的条数
128.     posx=NMEA_Comma_Pos(p1,3);    //得到可见北斗卫星总数
129.     if(posx!=0XFF)gpsx->beidou_svnum=NMEA_Str2num(p1+posx,&dx);
130.     for(i=0;i<len;i++)
131.     {
132.         p1=(u8*)strstr((const char *)p,"$BDGSV");
133.         for(j=0;j<4;j++)
134.         {
135.             posx=NMEA_Comma_Pos(p1,4+j*4);
136.             if(posx!=0XFF)gpsx->beidou_s1msg[slx].beidou_num=NMEA_Str2num(p1+posx,&dx);    //
            得到卫星编号
137.             else break;
138.             posx=NMEA_Comma_Pos(p1,5+j*4);

```

```

139.         if(posx!=0XFF)gpsx->beidou_slmsg[slx].beidou_eledeg=NMEA_Str2num(p1+posx,&dx);/
        /得到卫星仰角
140.         else break;
141.         posx=NMEA_Comma_Pos(p1,6+j*4);
142.         if(posx!=0XFF)gpsx->beidou_slmsg[slx].beidou_azideg=NMEA_Str2num(p1+posx,&dx);/
        /得到卫星方位角
143.         else break;
144.         posx=NMEA_Comma_Pos(p1,7+j*4);
145.         if(posx!=0XFF)gpsx->beidou_slmsg[slx].beidou_sn=NMEA_Str2num(p1+posx,&dx);    //
        得到卫星信噪比
146.         else break;
147.         slx++;
148.     }
149.     p=p1+1;//切换到下一个 BDGSV 信息
150. }
151. }
152.
153. //分析 GNGGA 信息
154. //gpsx:nmea 信息结构体
155. //buf:接收到的 GPS 数据缓冲区首地址
156. void NMEA_GNGGA_Analysis(nmea_msg *gpsx,u8 *buf)
157. {
158.     u8 *p1,dx;
159.     u8 posx;
160.     p1=(u8*)strstr((const char *)buf,"$GNGGA");
161.     posx=NMEA_Comma_Pos(p1,6);                                //得到 GPS 状态
162.     if(posx!=0XFF)gpsx->gpssta=NMEA_Str2num(p1+posx,&dx);
163.     posx=NMEA_Comma_Pos(p1,7);                                //得到用于定位的卫星数
164.     if(posx!=0XFF)gpsx->posslnum=NMEA_Str2num(p1+posx,&dx);
165.     posx=NMEA_Comma_Pos(p1,9);                                //得到海拔高度
166.     if(posx!=0XFF)gpsx->altitude=NMEA_Str2num(p1+posx,&dx);
167. }
168.
169. //分析 GNGSA 信息
170. //gpsx:nmea 信息结构体
171. //buf:接收到的 GPS 数据缓冲区首地址
172. void NMEA_GNGSA_Analysis(nmea_msg *gpsx,u8 *buf)
173. {
174.     u8 *p1,dx;
175.     u8 posx;
176.     u8 i;
177.     p1=(u8*)strstr((const char *)buf,"$GNGSA");
178.     posx=NMEA_Comma_Pos(p1,2);                                //得到定位类型
179.     if(posx!=0XFF)gpsx->fixmode=NMEA_Str2num(p1+posx,&dx);

```

```

180.     for(i=0;i<12;i++)                                //得到定位卫星编号
181.     {
182.         posx=NMEA_Comma_Pos(p1,3+i);
183.         if(posx!=0XFF)gpsx->possl[i]=NMEA_Str2num(p1+posx,&dx);
184.         else break;
185.     }
186.     posx=NMEA_Comma_Pos(p1,15);                        //得到 PDOP 位置精度因子
187.     if(posx!=0XFF)gpsx->pdop=NMEA_Str2num(p1+posx,&dx);
188.     posx=NMEA_Comma_Pos(p1,16);                        //得到 HDOP 位置精度因子
189.     if(posx!=0XFF)gpsx->hdop=NMEA_Str2num(p1+posx,&dx);
190.     posx=NMEA_Comma_Pos(p1,17);                        //得到 VDOP 位置精度因子
191.     if(posx!=0XFF)gpsx->vdop=NMEA_Str2num(p1+posx,&dx);
192. }
193.
194. //分析 GNRMC 信息
195. //gpsx:nmea 信息结构体
196. //buf:接收到的 GPS 数据缓冲区首地址
197. void NMEA_GNRMC_Analysis(nmea_msg *gpsx,u8 *buf)
198. {
199.     u8 *p1,dx;
200.     u8 posx;
201.     u32 temp;
202.     float rs;
203.     p1=(u8*)strstr((const char *)buf,"$GNRMC");//"$GNRMC", 经常有&和 GNRMC 分开的情况,故只判断
        GPRMC.
204.     posx=NMEA_Comma_Pos(p1,1);                        //得到 UTC 时间
205.     if(posx!=0XFF)
206.     {
207.         temp=NMEA_Str2num(p1+posx,&dx)/NMEA_Pow(10,dx);    //得到 UTC 时间,去掉 ms
208.         gpsx->utc.hour=temp/10000;
209.         gpsx->utc.min=(temp/100)%100;
210.         gpsx->utc.sec=temp%100;
211.     }
212.     posx=NMEA_Comma_Pos(p1,3);                        //得到纬度
213.     if(posx!=0XFF)
214.     {
215.         temp=NMEA_Str2num(p1+posx,&dx);
216.         gpsx->latitude=temp/NMEA_Pow(10,dx+2);    //得到°
217.         rs=temp%NMEA_Pow(10,dx+2);                //得到'
218.         gpsx->latitude=gpsx->latitude*NMEA_Pow(10,5)+(rs*NMEA_Pow(10,5-dx))/60;//转换为°
219.     }
220.     posx=NMEA_Comma_Pos(p1,4);                        //南纬还是北纬
221.     if(posx!=0XFF)gpsx->nshemi=(p1+posx);
222.     posx=NMEA_Comma_Pos(p1,5);                        //得到经度

```

```

223.     if(posx!=0XFF)
224.     {
225.         temp=NMEA_Str2num(p1+posx,&dx);
226.         gpsx->longitude=temp/NMEA_Pow(10,dx+2); //得到°
227.         rs=temp%NMEA_Pow(10,dx+2); //得到'
228.         gpsx->longitude=gpsx->longitude*NMEA_Pow(10,5)+(rs*NMEA_Pow(10,5-dx))/60; //转换
           为°
229.     }
230.     posx=NMEA_Comma_Pos(p1,6); //东经还是西经
231.     if(posx!=0XFF)gpsx->ewhemi=(p1+posx);
232.     posx=NMEA_Comma_Pos(p1,9); //得到 UTC 日期
233.     if(posx!=0XFF)
234.     {
235.         temp=NMEA_Str2num(p1+posx,&dx); //得到 UTC 日期
236.         gpsx->utc.date=temp/10000;
237.         gpsx->utc.month=(temp/100)%100;
238.         gpsx->utc.year=2000+temp%100;
239.     }
240. }
241.
242. //分析 GNVTG 信息
243. //gpsx:nmea 信息结构体
244. //buf:接收到的 GPS 数据缓冲区首地址
245. void NMEA_GNVTG_Analysis(nmea_msg *gpsx,u8 *buf)
246. {
247.     u8 *p1,dx;
248.     u8 posx;
249.     p1=(u8*)strstr((const char *)buf,"$GNVTG");
250.     posx=NMEA_Comma_Pos(p1,7); //得到地面速率
251.     if(posx!=0XFF)
252.     {
253.         gpsx->speed=NMEA_Str2num(p1+posx,&dx);
254.         if(dx<3)gpsx->speed*=NMEA_Pow(10,3-dx); //确保扩大 1000 倍
255.     }
256. }
257.
258. //提取 NMEA-0183 信息
259. //gpsx:nmea 信息结构体
260. //buf:接收到的 GPS 数据缓冲区首地址
261. void GPS_Analysis(nmea_msg *gpsx,u8 *buf)
262. {
263.     NMEA_GPGSV_Analysis(gpsx,buf); //GPGSV 解析
264.     NMEA_BDGSV_Analysis(gpsx,buf); //BDGSV 解析
265.     NMEA_GNGGA_Analysis(gpsx,buf); //GNGGA 解析

```



```

266.     NMEA_GNGSA_Analysis(gpsx,buf); //GPNSA 解析
267.     NMEA_GNRMCA_Analysis(gpsx,buf); //GPNMC 解析
268.     NMEA_GNVTG_Analysis(gpsx,buf); //GPNTG 解析
269. }
270.
271. //////////////////////////////////////////////////////////////////UBLOX           配           置           代           码
    //////////////////////////////////////////////////////////////////
272. //检查 CFG 配置执行情况
273. //返回值:0,ACK 成功
274. //      1,接收超时错误
275. //      2,没有找到同步字符
276. //      3,接收到 NACK 应答
277. u8 S1216F8BD_Cfg_Ack_Check(void)
278. {
279.     u16 len=0,i;
280.     u8 rval=0;
281.     while((USART3_RX_STA&0X8000)==0 && len<100)//等待接收到应答
282.     {
283.         len++;
284.         delay_ms(5);
285.     }
286.     if(len<100) //超时错误.
287.     {
288.         len=USART3_RX_STA&0X7FFF; //此次接收到的数据长度
289.         for(i=0;i<len;i++)
290.         {
291.             if(USART3_RX_BUF[i]==0X83)break;
292.             else if(USART3_RX_BUF[i]==0X84)
293.             {
294.                 rval=3;
295.                 break;
296.             }
297.         }
298.         if(i==len)rval=2; //没有找到同步字符
299.     }else rval=1; //接收超时错误
300.     USART3_RX_STA=0; //清除接收
301.     return rval;
302. }
303. //配置 S1216F8-BD_GPS/北斗模块波特率
304. //baud_id:0~8, 对应波特率,4800/9600/19200/38400/57600/115200/230400/460800/921600
305. //返回值:0,执行成功;其他,执行失败(这里不会返回 0 了)
306. u8 S1216F8BD_Cfg_Prt(u8 baud_id)
307. {
308.     S1216F8BD_baudrate *cfg_prt=(S1216F8BD_baudrate *)USART3_TX_BUF;

```

```

309.     cfg_prt->sos=0XA1A0;        //引导序列(小端模式)
310.     cfg_prt->PL=0X0400;          //有效数据长度(小端模式)
311.     cfg_prt->id=0X05;            //配置波特率的 ID
312.     cfg_prt->com_port=0X00;      //操作串口 1
313.     cfg_prt->Baud_id=baud_id;     ////波特率对应编号
314.     cfg_prt->Attributes=1;        //保存到 SRAM&FLASH
315.     cfg_prt->CS=cfg_prt->id^cfg_prt->com_port^cfg_prt->Baud_id^cfg_prt->Attributes;
316.     cfg_prt->end=0X0A0D;          //发送结束符(小端模式)
317.     S1216F8BD_Send_Date((u8*)cfg_prt,sizeof(S1216F8BD_baudrate));//发送数据给 S1216F8BD
318.     delay_ms(200);               //等待发送完成
319.     USART3_Init(BAUD_id[baud_id]); //重新初始化串口 2
320.     return S1216F8BD_Cfg_Ack_Check();//这里不会返回 0,因为 UBLOX 发回来的应答在串口重新初始化的时
    候已经被丢弃了。
321. }
322. //配置 S1216F8BD_GPS 模块的时钟脉冲宽度
323. //width:脉冲宽度 1~100000(us)
324. //返回值:0,发送成功;其他,发送失败。
325. u8 S1216F8BD_Cfg_Tp(u32 width)
326. {
327.     u32 temp=width;
328.     S1216F8BD_pps_width *cfg_tp=(S1216F8BD_pps_width *)USART3_TX_BUF;
329.     temp=(width>>24)|((width>>8)&0X000FF00)|((width<<8)&0X00FF0000)|((width<<24)&0XFF00000
    0);//小端模式
330.     cfg_tp->sos=0XA1A0;          //cfg header(小端模式)
331.     cfg_tp->PL=0X0700;          //有效数据长度(小端模式)
332.     cfg_tp->id=0X65;            //cfg tp id
333.     cfg_tp->Sub_ID=0X01;        //数据区长度为 20 个字节。
334.     cfg_tp->width=temp;         //脉冲宽度,us
335.     cfg_tp->Attributes=0X01;    //保存到 SRAM&FLASH
336.     cfg_tp->CS=cfg_tp->id^cfg_tp->Sub_ID^(cfg_tp->width>>24)^(cfg_tp->width>>16)&0XFF^(cfg_
    tp->width>>8)&0XFF^cfg_tp->width&0XFF^cfg_tp->Attributes;    //用户延时为 0ns
337.     cfg_tp->end=0X0A0D;          //发送结束符(小端模式)
338.     S1216F8BD_Send_Date((u8*)cfg_tp,sizeof(S1216F8BD_pps_width));//发送数据给 NEO-6M
339.     return S1216F8BD_Cfg_Ack_Check();
340. }
341. //配置 S1216F8BDF8-BD 的更新速率
342. //Frep: (取值范围:1,2,4,5,8,10,20,25,40,50) 测量时间间隔, 单位为 Hz, 最大不能大于 50Hz
343. //返回值:0,发送成功;其他,发送失败。
344. u8 S1216F8BD_Cfg_Rate(u8 Frep)
345. {
346.     S1216F8BD_PosRate *cfg_rate=(S1216F8BD_PosRate *)USART3_TX_BUF;
347.     cfg_rate->sos=0XA1A0;        //cfg header(小端模式)
348.     cfg_rate->PL=0X0300;        //有效数据长度(小端模式)
349.     cfg_rate->id=0X0E;          //cfg rate id

```

```

350.     cfg_rate->rate=Frep;           //更新速率
351.     cfg_rate->Attributes=0X01;      //保存到 SRAM&FLASH .
352.     cfg_rate->CS=cfg_rate->id^cfg_rate->rate^cfg_rate->Attributes;//脉冲间隔,us
353.     cfg_rate->end=0X0A0D;          //发送结束符(小端模式)
354.     S1216F8BD_Send_Date((u8*)cfg_rate,sizeof(S1216F8BD_PosRate));//发送数据给 NEO-6M
355.     return S1216F8BD_Cfg_Ack_Check();
356. }
357. //发送一批数据给 S1216F8BDF8-BD, 这里通过串口 2 发送
358. //dbuf: 数据缓存首地址
359. //len: 要发送的字节数
360. void S1216F8BD_Send_Date(u8* dbuf,u16 len)
361. {
362.     u16 j;
363.     for(j=0;j<len;j++)//循环发送数据
364.     {
365.         while((USART3->SR&0X40)==0);//循环发送,直到发送完毕
366.         USART3->DR=dbuf[j];
367.     }
368. }

```

这部分代码可以分为 2 个部分，第一部分是 NMEA-0183 数据解析部分，另外一部分则是 S1216F8BDF8-BD 协议控制部分。

NMEA-0183 协议解析部分，这里利用了一个简单的数逗号方法来解析。我们知道 NMEA-0183 协议都是以类似\$GPGSV 的开头，然后固定输出格式，不论是否有数据输出，逗号是肯定会有有的，而且都会以 ‘ * ’ 作为有效数据的结尾，所以，我们了解了 NMEA-0183 协议的数据格式之后，就可以通过数逗号的方法，来解析数据了。本代码实现了对 NMEA-0183 协议的\$GNGGA、\$GPGSA、\$GNGSV、\$BDGSV、\$GNRMC 和\$GNVTG 等 6 类帧的解析，结果存放在通过 gps.h 定义的 nmea_msg 结构体内。

S1216F8BDF8-BD 协议控制部分，此部分我们只实现了 S1216F8BDF8-BD 模组常用的 3 个配置：串口波特率设置、 PPS 输出脉冲宽度设置、输出频率设置。

串口波特率设置，通过函数 S1216F8BD_Cfg_Prt 实现，该函数可以设置模块的波特率。

PPS 输出脉冲宽度设置，通过函数 S1216F8BD_Cfg_Tp 实现，可以设置脉冲宽度（ 1us~100ms ）。

输出频率设置，通过函数 S1216F8BD_Cfg_Rate 实现，该函数可以设置模块的测量输出频率，最快可以达到 20Hz 的测量输出频率。

最后 S1216F8BD_Send_Date 函数，用于发送一批设置好的数据给串口 3，完成对 GPS 模块的配置。

gps.h 里面的代码我们也不列出了，请大家参考对应源码。

3.3 main.c

最后我们打开 main.c 文件，里面的代码如下：

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "key.h"
5.  #include "usart.h"
6.  #include "tftlcd.h"
7.  #include "gps.h"
8.  #include "usart3.h"
9.  #include "string.h"
10.
11.
12.
13.  u8 USART1_TX_BUF[USART3_MAX_RECV_LEN];           //串口 1,发送缓存区
14.  nmea_msg gpsx;                                     //GPS 信息
15.  __align(4) u8 dtbuf[50];                           //打印缓存器
16.  const u8*fixmode_tbl[4]={"Fail","Fail"," 2D "," 3D "}; //fix mode 字符串
17.
18.  //显示 GPS 定位信息
19.  void Gps_Msg_Show(void)
20.  {
21.      float tp;
22.      FRONT_COLOR=BLUE;
23.      tp=gpsx.longitude;
24.      sprintf((char *)dtbuf,"Longitude:%.5f %1c ",tp/=100000,gpsx.ewhemi); //得到经度字符
    串
25.      LCD_ShowString(10,120,200,16,16,dtbuf);
26.      tp=gpsx.latitude;
27.      sprintf((char *)dtbuf,"Latitude:%.5f %1c ",tp/=100000,gpsx.nshemi); //得到纬度字符
    串
28.      LCD_ShowString(10,140,200,16,16,dtbuf);
29.      tp=gpsx.altitude;
30.      sprintf((char *)dtbuf,"Altitude:%.1fm ",tp/=10); //得到高度字符
    串
31.      LCD_ShowString(10,160,200,16,16,dtbuf);
32.      tp=gpsx.speed;
```

```

33.     sprintf((char *)dtbuf, "Speed: %.3fkm/h", tp/=1000); //得到速度字符串
34.     LCD_ShowString(10,180,200,16,16,dtbuf);
35.     if(gpsx.fixmode<=3) //定位状态
36.     {
37.         sprintf((char *)dtbuf, "Fix Mode:%s", fixmode_tbl[gpsx.fixmode]);
38.         LCD_ShowString(10,200,200,16,16,dtbuf);
39.     }
40.     sprintf((char *)dtbuf, "GPS+BD Valid satellite:%02d", gpsx.posslnum); //用于定位的 GPS 卫星数
41.     LCD_ShowString(10,220,200,16,16,dtbuf);
42.     sprintf((char *)dtbuf, "GPS Visible satellite:%02d", gpsx.svnum%100); //可见 GPS 卫星数
43.     LCD_ShowString(10,240,200,16,16,dtbuf);
44.
45.     sprintf((char *)dtbuf, "BD Visible satellite:%02d", gpsx.beidou_svnum%100); //可见北斗卫星数
46.     LCD_ShowString(10,260,200,16,16,dtbuf);
47.
48.     sprintf((char *)dtbuf, "UTC Date:%04d/%02d/%02d", gpsx.utc.year, gpsx.utc.month, gpsx.utc.date); //显示 UTC 日期
49.     LCD_ShowString(10,280,200,16,16,dtbuf);
50.     sprintf((char *)dtbuf, "UTC Time:%02d:%02d:%02d", gpsx.utc.hour, gpsx.utc.min, gpsx.utc.sec); //显示 UTC 时间
51.     LCD_ShowString(10,300,200,16,16,dtbuf);
52. }
53.
54.
55.
56. int main()
57. {
58.     u16 i, rxlen;
59.     u16 lenx;
60.     u8 key=0XFF;
61.     u8 upload=0;
62.
63.     SysTick_Init(168);
64.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
65.     LED_Init();
66.     KEY_Init();
67.     USART1_Init(9600);
68.     TFTLCD_Init(); //LCD 初始化
69.     USART3_Init(38400);
70.

```

```

71.     FRONT_COLOR=RED;
72.     LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,16,"PRECHIN");
73.     LCD_ShowString(10,30,tftlcd_data.width,tftlcd_data.height,16,"www.prechin.com");
74.     LCD_ShowString(10,50,tftlcd_data.width,tftlcd_data.height,16,"S1216F8-BD GPS Test");
75.     LCD_ShowString(10,70,tftlcd_data.width,tftlcd_data.height,16,"K_UP:Upload NMEA Data SW")
    ;
76.     LCD_ShowString(10,90,tftlcd_data.width,tftlcd_data.height,16,"NMEA Data Upload:OFF");
77.
78.
79.     if(S1216F8BD_Cfg_Rate(5)!=0)    //设置定位信息更新速度为 5Hz,顺便判断 GPS 模块是否在位。
80.     {
81.         LCD_ShowString(10,120,200,16,16,"S1216F8-BD GPS Setting...");
82.         do
83.         {
84.             USART3_Init(9600);    //初始化串口 3 波特率为 9600
85.             S1216F8BD_Cfg_Prt(3);    //重新设置模块的波特率为 38400
86.             USART3_Init(38400);    //初始化串口 3 波特率为 38400
87.             key=S1216F8BD_Cfg_Tp(100000);    //脉冲宽度为 100ms
88.         }while(S1216F8BD_Cfg_Rate(5)!=0&&key!=0);//配置 S1216F8BDF8-BD 的更新速率为 5Hz
89.         LCD_ShowString(10,120,200,16,16,"S1216F8-BD GPS Set Done!!");
90.         delay_ms(500);
91.         LCD_Fill(10,120,10+200,120+16,WHITE);//清除显示
92.     }
93.     while(1)
94.     {
95.         delay_ms(1);
96.         if(USART3_RX_STA&0X8000)    //接收到一次数据了
97.         {
98.             rxlen=USART3_RX_STA&0X7FFF; //得到数据长度
99.             for(i=0;i<rxlen;i++)USART1_TX_BUF[i]=USART3_RX_BUF[i];
100.             USART3_RX_STA=0;    //启动下一次接收
101.             USART1_TX_BUF[i]=0;    //自动添加结束符
102.             GPS_Analysis(&gpsx,(u8*)USART1_TX_BUF);//分析字符串
103.             Gps_Msg_Show();    //显示信息
104.             if(upload)printf("\r\n%s\r\n",USART1_TX_BUF);//发送接收到的数据到串口 1
105.         }
106.         key=KEY_Scan(0);
107.         if(key==KEY_UP_PRESS)
108.         {
109.             upload=!upload;
110.             FRONT_COLOR=RED;
111.             if(upload)LCD_ShowString(10,90,200,16,16,"NMEA Data Upload:ON ");
112.             else LCD_ShowString(10,90,200,16,16,"NMEA Data Upload:OFF");
113.         }

```

```

114.         if((lenx%500)==0)
115.             LED1=!LED1;
116.             lenx++;
117.     }
118. }

```

此部分代码比较简单，main 函数初始化硬件之后，通过 S1216F8BD_Cfg_Rate 函数判断模块是否在位，如果不在位，则尝试去设置模块的波特率为 38400，直到检测到模块在位为止。然后进入死循环，等待串口 3 接收 GPS/北斗数据。每次接收到 GPS/北斗模块发送过来的数据，就执行数据解析，数据解析后执行 GPS/北斗定位数据的显示，并可以根据需要（通过 KEY_UP 按键开启/关闭），将收到的数据通过串口 1 发送给上位机。

4 实验现象

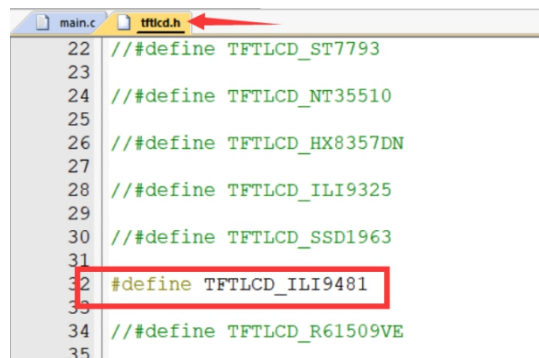
首先，请先确保硬件都已经连接好：

①PZ-S1216F8-BD GPS 模块与 F407 最小系统&天马&麒麟开发板连接（可参考硬件设计小节）

②开发板插上 TFTLCD 液晶，对于没有 TFTLCD 的用户，可通过串口输出。

③使用 USB 线给开发板供电。

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：

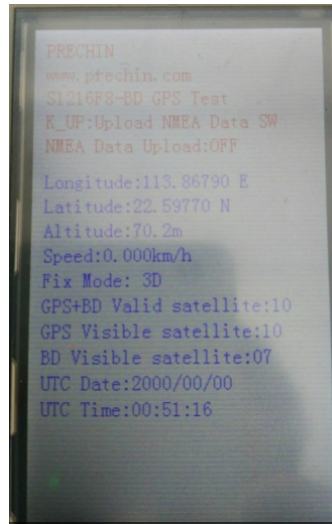


```

22 // #define TFTLCD_ST7793
23
24 // #define TFTLCD_NT35510
25
26 // #define TFTLCD_HX8357DN
27
28 // #define TFTLCD_ILI9325
29
30 // #define TFTLCD_SSD1963
31
32 #define TFTLCD_ILI9481
33
34 // #define TFTLCD_R61509VE
35

```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以麒麟开发板为展示）LCD 界面显示如下图所示：



上图是我们的 GPS/北斗模块成功定位后的照片，可以得到当前地点的经纬度、高度、速度、定位模式、用于定位卫星数、可见卫星数和 UTC 日期时间等信息。此时我们的 PZ-S1216F8-BD GPS/北斗模块，用于定位的卫星达到 10 颗，可见的 GPS 和北斗卫星一共 17 颗。

5 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）